BPSC TRE 4.0

Computer Science (Class 11–12)

Day 2

I. Computer Fundamentals > Software & Language Translators

2026-10-29

BPSC TRE 4.0 Computer Science — Day 2 — I. Computer Fundamentals > Software & Language Translators — 2026-10-29

Notes • 30 Questions • Answer Key + व्याख्या

Part 1 — Software क्या है + System Software

① पढ़ने से पहले

- आज के हिस्से: (1) Software और System Software, (2) Programming tools, Application software, Firmware, (3) Software licence, (4) Programming languages और generations, (5) Assembler, Compiler, Interpreter, (6) Linker, Loader और C++ program का सफ़र, (7) Compile vs Interpret example, (8-10) NET level: compiler phases, assembler/loader/macro, bootstrapping/JIT (Just-In-Time), फिर Numericals और Abbreviations.
- Source: BSEB (Bihar School Examination Board) Class II Computer Science (C++ based) + NCERT (National Council of Educational Research and Training) Class XI CS Ch 1 (1.7 Software). NET (National Eligibility Test) वाला हिस्सा '(SCERT/NCERT से आगे, NET level)' लिखकर अलग किया है. SCERT = State Council of Educational Research and Training.
- BPSC (Bihar Public Service Commission) TRE (Teacher Recruitment Exam) 1.0 / 2.0 / 3.0 के इस topic वाले 10 असली सवाल अब Questions हिस्से में सबसे पहले हैं, हर option की व्याख्या के साथ.

② I. आसान भाषा में

- Hardware = computer का वो हिस्सा जिसे छू सकते हैं: CPU (Central Processing Unit), RAM (Random Access Memory), keyboard, monitor.
- Software = instructions (निर्देश) और data का set जो hardware से काम करवाता है. इसे छू नहीं सकते.
- Program = किसी एक काम के लिए instructions का क्रम से लिखा

group. Programs + data + documentation = software.

- NCERT: software, user और hardware के बीच interface (पुल) है. बिना software के hardware बेकार है.
- NCERT के अनुसार software के 3 प्रकार: (i) System software (ii) Programming tools (iii) Application software.

(3) 2. रोज़मर्रा का example

- एक bus सोचो:
  - → Bus का engine, पहिए, सीट = hardware.
  - → Driver जो bus चलाता है, सबको संभालता है = system software (OS (Operating System)).
  - → Conductor की ticket machine, जो यात्री के काम आती है = application software.
  - → Driver को रास्ता बताने वाला नक्शा (instructions का set) = program.

(4) 3. System Software के हिस्से step by step

- System software: computer को चलाने वाली basic software, सीधे hardware से बात करती है और user या दूसरी software को सेवा (platform) देती है.
- (A) OS (Operating System): सबसे basic system software, इसके बिना कोई दूसरी software नहीं चलती. Resource manager है: CPU, RAM, disk, I/O (Input/Output). Example: Windows, Linux, Ubuntu, Android.
- (B) System utilities: computer की देखभाल (maintenance) और setting (configuration) के लिए. OS के साथ आने वाली: disk defragmenter, formatting utility, system restore, Scandisk. अलग से लगने वाली: anti-virus, disk cleaner, compression tool, backup.
- (C) Device drivers: OS और किसी एक device (printer, scanner) के बीच translator / mediator. हर नए device का अपना driver होता है.
- (D) Language translators (assembler, compiler, interpreter) +

linker, loader: NET level पर इन्हें भी system software गिनते हैं. NCERT इन्हें 'programming tools' में रखता है.

5 4. Exam Point + Trap

- 'Group of instructions that directs a computer' = program (memory या storage नहीं).
- Scandisk, Format = system tools. Folder कोई tool नहीं, सिर्फ files रखने का डिब्बा (container) है.
- Trap: device driver को application software मत समझना, यह system software है.
- NCERT: application software के बिना computer चल सकता है, पर system software (OS) के बिना नहीं.

6 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- Type 1 — Definition (TRE 2.0 Q107): 'Group of instructions that directs a computer is called' -> memory / storage / program. जवाब program. Trick: memory और storage सिर्फ रखने की जगह हैं, निर्देश देने वाला program है.
- Type 2 — 'Which is NOT a system tool' (TRE 2.0 Q84): Scandisk, Format, Folder. जवाब Folder. Trick: हर option के आगे लिखो 'यह कोई काम (tool) करता है?' हाँ/ना.
- Related (Programming topic में गिना, TRE 2.0 Q108): 'A computer program consists of' -> algorithms written in computer language.

7 6. NET level में ऐसे आता है

- NET में software classification सीधे कम आता है, ज़्यादातर 'Which is NOT system software' या match the following (software -> category).
- Mini example: 'Which is NOT system software? (a) Device driver (b) Defragmenter (c) OS (d) Spreadsheet' -> (d) Spreadsheet = application software.

⑧ 7. याद रखो

- Software = instructions; Program = एक काम के instructions; System software = OS + utilities + drivers (+ translators).
- Folder tool नहीं; Scandisk/Format tool हैं.

Programming Tools, Application Software, Firmware

① 1. आसान भाषा में

- Programming tools: program लिखने, बदलने और ठीक करने के software. जैसे languages, translators, editor, debugger, IDE (Integrated Development Environment).
- Application software: user का कोई खास काम करने वाला software (letter लिखना, हिसाब करना). यह system software के ऊपर चलता है.
- Firmware: hardware के अंदर ROM (Read Only Memory) / flash में हमेशा के लिए रखा software. जैसे BIOS (Basic Input Output System), UEFI (Unified Extensible Firmware Interface).

② 2. रोज़मर्रा का example

- दर्ज़ी (tailor) की दुकान:
  - → कैंची, सुई, फीता = programming tools (बनाने के औज़ार).
  - → दुकान पर टंगे ready-made कपड़े = general purpose software (सबके लिए, जैसे MS Word).
  - → नाप लेकर सिला कपड़ा = customised (tailor-made) software (एक school या company के लिए). यही example NCERT ने दिया है.
  - → Sewing machine के अंदर लगा छोटा control chip = firmware.

③ 3. हर प्रकार step by step

- Editor: source code type करके file में save करने का software (Notepad, Python IDLE (Integrated Development and Learning Environment) का editor).
- Debugger: program की गलतियाँ (errors / bugs) ढूँढने और ठीक करने का software.
- IDE = editor + build tools (translator) + debugger एक जगह. Example: Turbo C++, Code::Blocks, Dev-C++, NetBeans, Eclipse, Python IDLE.
- General purpose application: word processor, spreadsheet (LibreOffice Calc), presentation, browser (Mozilla), image editor (GIMP, Photoshop).
- Customised application: school management, accounting software, किसी company की website.
- Firmware: BIOS / UEFI, washing machine, TV remote, router का control program. Power on होते ही BIOS POST (Power-On Self-Test) करता है और bootstrap loader OS (Operating System) को hard disk से RAM (Random Access Memory) में लाता है (NCERT 'Think and Reflect' का जवाब).
- Middleware (SCERT/NCERT से आगे, NET level): OS और applications के बीच 'गोंद (glue)' software, जैसे database connectivity, message queue.

④ 4. Exam Point + Trap

- Application software बिना system software के नहीं चल सकता; system software अकेले चल सकता है.
- 'Software stored in ROM' दिखे तो firmware. Firmware = hardware + software का मेल.
- Trap: Freeware (licence का प्रकार) और Firmware (ROM में रखा software) अलग हैं, spelling मिलती-जुलती है.

- Trap: IDE कोई language नहीं है, वो tools का package है.

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- इस sub-topic पर TRE 1.0/2.0/3.0 में सीधा सवाल नहीं आया.
- Related (Hardware topic में गिना, TRE 2.0 Q77): '_____ is used to store softwares that does not update' -> SRAM / DRAM / ROM. जवाब ROM, यानी firmware की जगह.
- IDE वाला Turbo C++ shortcut सवाल (TRE 1.0 Q50) Part 6 में है.

⑥ 6. NET level में ऐसे आता है

- Definition type: 'Software permanently stored in ROM/flash' -> Firmware.
- Comparison type: firmware vs freeware vs middleware; editor vs debugger vs IDE.
- Mini example: 'IDE consists of: (a) editor only (b) editor + build tools + debugger' -> (b).

⑦ 7. याद रखो

- IDE = editor + translator + debugger. Application software = system software के ऊपर. Firmware = ROM में, BIOS.
- General purpose = ready-made कपड़ा; Customised = नाप का सिला कपड़ा.

Software Licence — Proprietary, Freeware, Shareware, FOSS

① 1. आसान भाषा में

- Licence = software बनाने वाले की शर्तें: तुम software के साथ क्या कर सकते हो और क्या नहीं.
- Source code = programmer का लिखा असली program. यह मिले तो

software को पढ़ और बदल सकते हैं.

- NCERT 1.7.5: एक ही software freeware, open source या proprietary हो सकता है, यह बनाने वाले की शर्तों पर निर्भर है.

② 2. रोज़मर्रा का example

- मिठाई की दुकान:
  - → पैसे देकर मिठाई खरीदो, पर recipe नहीं मिलेगी = Proprietary.
  - → मिठाई मुफ़्त में चखो, पर recipe नहीं मिलेगी = Freeware.
  - → एक हफ़्ता मुफ़्त, फिर पैसे दो = Shareware.
  - → मिठाई भी और recipe भी सबके लिए खुली, बदलकर आगे बाँटो = FOSS (Free and Open Source Software).

③ 3. हर प्रकार step by step

- Proprietary software: copyright vendor के पास, खरीदना पड़ता है, source code नहीं मिलता. शर्तें EULA (End User License Agreement) में. Example (NCERT): Microsoft Windows, Tally, Quick Heal.
- Freeware: use free, पर source code नहीं मिलता, बदल नहीं सकते. Example (NCERT): Skype, Adobe Reader.
- Shareware: 'try before you buy', trial free, बाद में पैसे; अक्सर कुछ features बंद. Example: WinRAR (trial).
- FOSS: source code freely available, कोई भी पढ़/बदल/बाँट सकता है. Example (NCERT): Ubuntu/Linux, Python, LibreOffice, OpenOffice, Mozilla Firefox.
  - → 'Free' का मतलब freedom (आज़ादी), 'मुफ़्त' ज़रूरी नहीं. FSF (Free Software Foundation) की 4 freedoms: run, study, share, modify.
- Public domain: कोई copyright नहीं, कोई भी कुछ भी कर सकता है.
- IPR (Intellectual Property Right): copyright, patent, trademark से software की रक्षा (NCERT Ch 11). बिना अनुमति copy करना = software piracy.
- GNU GPL (GNU's Not Unix General Public License): run, study,

share, modify की freedom + copyleft: बदला हुआ version बाँटो तो वो भी GPL में, source के साथ. NCERT: GPL software की copies बाँटने पर fee ले सकते हैं.

- CC (Creative Commons): website, music, film, किताब जैसे creative काम के लिए licence (software के लिए GPL).
- Permissive licences (SCERT/NCERT से आगे, NET level): MIT (Massachusetts Institute of Technology), BSD (Berkeley Software Distribution), Apache. बदला code बंद (closed) करना भी allowed, copyleft नहीं.

④ 4. Exam Point + Trap

- Trap: Freeware और open source एक नहीं. Freeware = दाम free, source बंद. Open source = source खुला.
- 'Copyleft' दिखे = GPL. 'Trial then pay' = Shareware. 'Source + modify allowed' = Open source.
- Trap: GPL software बेच सकते हैं (fee ले सकते हैं), बस source देना होगा.

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- इस sub-topic पर TRE 1.0/2.0/3.0 में सीधा सवाल नहीं आया.
- Related (Cyber ethics topic में गिना, TRE 2.0 Q119): 'Copying software without the permission of its authors' -> larceny / software piracy / highway robbery. जवाब software piracy.

⑥ 6. NET level में ऐसे आता है

- Match type: Shareware -> trial; Freeware -> free but closed; GPL -> copyleft; CC -> creative works.
- Statement type: 'I. Freeware का source code मिलता है II. GPL में modified version भी GPL होगा' -> I गलत, II सही.

⑦ 7. याद रखो

- मिठाई वाला trick: खरीदो (Proprietary), चखो (Freeware), हफ़्ता मुफ़्त (Shareware), recipe भी (FOSS).
- GPL = copyleft + fee allowed. CC = creative works.

Programming Languages — Low/High Level और Generations

① 1. आसान भाषा में

- Computer सिर्फ 0 और 1 समझता है. इंसान के लिए 0/1 में लिखना बहुत मुश्किल, इसलिए अलग-अलग programming languages बनीं.
- NCERT: दो बड़े प्रकार: Low-level languages (machine dependent) और High-level languages, HLL (High-Level Language) (machine independent).

② 2. रोज़मर्रा का example

- किसी विदेशी से बात करनी है:
  - → उसकी अपनी भाषा में बोलो = machine language (सबसे तेज़ समझ, पर तुम्हारे लिए बहुत मुश्किल).
  - → इशारों के छोटे code-words (MOV, ADD) = assembly language.
  - → अपनी भाषा में बोलो और translator बीच में बैठा हो = high-level language + compiler/interpreter.

③ 3. हर प्रकार step by step

- Machine language: सिर्फ 0/1. CPU (Central Processing Unit) सीधे समझता है, सबसे fast. पर लिखना, पढ़ना, गलती ढूँढना बहुत मुश्किल; opcodes और addresses याद रखने पड़ते हैं.
- Assembly language: mnemonics (MOV, ADD, SUB, JMP) जैसे English-जैसे छोटे शब्द. पढ़ना आसान, पर हर CPU की अलग (portable

नहीं). Assembler से machine code बनती है.

- HLL: English जैसे statements, अपने grammar (syntax) rules. Machine independent. Compiler/interpreter चाहिए. Example: C++, Java, Python, Pascal.
- Generations, GL (Generation Language):
  - → 1GL = machine language (binary), 1940s-50s.
  - → 2GL = assembly language (mnemonics + symbolic addresses).
  - → 3GL = procedural HLL: FORTRAN (FORmula TRANslation, 1957, IBM (International Business Machines), पहली प्रमुख HLL), COBOL (COmmon Business Oriented Language), BASIC (Beginner's All-purpose Symbolic Instruction Code), Pascal, C, C++, Java, Python.
  - → 4GL = non-procedural: 'क्या चाहिए' बताओ, 'कैसे' नहीं. SQL (Structured Query Language), report generators, MATLAB (MATrix LABoratory).
  - → 5GL = logic/constraints देकर हल: Prolog (PROgramming in LOGic), Mercury; AI (Artificial Intelligence), expert systems.
- Pascal: Niklaus Wirth (1970) ने structured programming सिखाने के लिए बनाई: block structure, if / while / repeat.

④ 4. Exam Point + Trap

- Speed: machine > assembly > HLL. Portability और आसानी: HLL > assembly > machine.
- Translator: machine -> कोई नहीं; assembly -> assembler; HLL -> compiler / interpreter.
- Trap: 'Machine level language is a ...' का जवाब low level language, 'assembly language' नहीं.
- Trap: SQL 4GL है, 3GL नहीं. Prolog 5GL है.
- HLL के फायदे: portability, आसान development, readability, maintenance. नुकसान: low-level से थोड़ा कम efficient.

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- Type — 'Which language is better for ...' (TRE 1.0 Q42): 'better for writing structured code' -> PASCAL / FORTRAN / BASIC. जवाब PASCAL.
- Solve कैसे: हर language का मकसद याद रखो: FORTRAN = scientific गणित, COBOL = business, BASIC = शुरुआती लोग, Pascal = structured programming सिखाना.

⑥ 6. NET level में ऐसे आता है

- Direct type (NET Dec 2022): 'Machine Level Language is a/an' -> Low level language.
- Match type: 1GL-binary, 2GL-assembly, 3GL-C++, 4GL-SQL, 5GL-Prolog.
- Comparison: 'Which is machine dependent?' -> assembly; 'Which needs no translator?' -> machine language.

⑦ 7. याद रखो

- 1-0/1, 2-Assembly, 3-C++/Java, 4-SQL, 5-Prolog. Pascal = structured. FORTRAN = पहली HLL (1957).
- Low level = machine dependent; High level = machine independent.

Language Translators — Assembler, Compiler, Interpreter

① 1. आसान भाषा में

- Computer सिर्फ machine language समझता है. Assembly या HLL (High-Level Language) में लिखा source code -> translator -> object code / machine code.

- NCERT के 3 translators: assembler, compiler, interpreter.
- Source code (.cpp, .py) इंसान के लिए; object code (.obj / .o) machine के लिए.

② 2. रोज़मर्रा का example

- एक अंग्रेज़ी किताब हिंदी में चाहिए:
  - → पूरी किताब एक बार में translate करके छपी हिंदी किताब दे दो = Compiler (बाद में translator की ज़रूरत नहीं).
  - → मंच पर भाषण के साथ-साथ एक-एक वाक्य बोलकर translate करो = Interpreter (हर बार translator चाहिए).
  - → Short-hand (code words) को पूरे शब्दों में बदलो = Assembler.

③ 3. हर translator step by step

- Assembler: assembly language -> machine language. Mnemonic -> opcode (ADD -> binary code), labels -> addresses. हर assembler एक specific processor के instruction set के लिए, इसलिए machine code portable नहीं (NCERT).
- Compiler: पूरा HLL program एक साथ (as a whole) पढ़कर object code बनाता है. Errors की list पूरी translation के बाद. Input = source program, output = object code. एक बार translate हो गया तो compiler की ज़रूरत नहीं (NCERT). Example: C++ के g++, Turbo C++; C, FORTRAN (FORmula TRANslation), COBOL (COmmon Business Oriented Language).
- Interpreter: एक line (statement) translate करो -> तुरंत चलाओ -> अगली line. Error मिलते ही वहीं रुकता है. Object file नहीं बनती, इसलिए हर बार चलाने पर interpreter चाहिए (NCERT). Example: Python, BASIC (Beginner's All-purpose Symbolic Instruction Code), JavaScript, Ruby.
- Compiler vs Interpreter:
  - → Unit: compiler = पूरा program; interpreter = एक statement.
  - → Speed: compiled program तेज़; interpreted धीमा (हर बार

translation).
→Object code: compiler बनाता है (.exe); interpreter नहीं.
→Errors: compiler = सब errors एक साथ अंत में; interpreter = पहली error पर रुकता है, इसलिए debugging आसान.
→Memory: compiler को translation में ज़्यादा; interpreter को कम.
→Source security: compiled .exe बाँटो, source छुपा रहता है; interpreted में source देना पड़ता है.

④ 4. Exam Point + Trap

- 'Assembly -> machine' = assembler. 'HLL -> machine, whole program' = compiler. 'Statement-by-statement' = interpreter.
- Trap: options में linker और loader भी आते हैं, ये translator नहीं हैं. Linker files जोड़ता है, loader memory में लाता है.
- Trap: 'Compiler के बारे में true' type में तीनों statements सही हों तो जवाब More than one of the above.

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- Type 1 — Definition (TRE 2.0 Q79, Q80, Q81): एक ही ढाँचे के 3 सवाल: 'assembly -> machine' = assembler; 'HLL -> equivalent machine program' = compiler; 'statement-by-statement' = interpreter. Solve: सवाल में keyword ढूँढो: assembly / whole / statement.
- Type 2 — True statement, multiple correct (TRE 1.0 Q44): (A) input source program (B) whole translation into object code (C) output object code. तीनों सही -> (D) More than one of the above. Trick: हर option अलग से हाँ/ना करो; 2 या ज़्यादा हाँ = D.

⑥ 6. NET level में ऐसे आता है

- Comparison/statement type: 'I. Interpreter object code बनाता है II. Compiled code तेज़ चलता है -> I गलत, II सही.
- Which is true type: 'Interpreter is needed every time the program

runs' -> सही.

7 7. याद रखो

- Assembler = short-hand खोलना; Compiler = पूरी किताब; Interpreter = भाषण का साथ-साथ अनुवाद.
- Linker और loader translator नहीं हैं.

Linker, Loader और C++ Program का पूरा सफ़र

1 1. आसान भाषा में

- Compiler से बनी object file अकेले नहीं चलती. Linker उसे library (जैसे iostream के cout) से जोड़कर executable (.exe) बनाता है.
- Loader उस .exe को hard disk से main memory (RAM (Random Access Memory)) में लाकर CPU (Central Processing Unit) से चलवाता है. Loader OS (Operating System) का हिस्सा है.

2 2. रोज़मर्रा का example

- Rail गाड़ी बनाना:
  - →अलग-अलग factory में बने डिब्बे = object files.
  - →Yard में डिब्बों को engine से जोड़ना = Linker.
  - →तैयार गाड़ी को platform पर लाकर खड़ा करना ताकि चल सके = Loader.

3 3. C++ program step by step (hello.cpp से output तक)

- Step 1 Preprocessor: # वाली lines: #include (header file की text चिपकाना), #define (macro substitution).
- Step 2 Compiler: preprocessed code -> assembly/object code. Syntax errors (semicolon missing) यहीं पकड़े जाते हैं.

- Step 3 Assembler: assembly -> object file (.obj / .o).
- Step 4 Linker: object + libraries -> .exe. 'undefined reference' जैसी errors यहाँ.
  - → Symbol resolution: एक file में call हुआ function दूसरी file/library में कहाँ है, वो address जोड़ना. Linker symbolic references को addresses में बदलता है (उल्टा नहीं).
  - → Static linking: library code .exe में copy (बड़ी file). Dynamic linking: run time पर shared library जुड़ती है: DLL (Dynamic Link Library) Windows में, .so (shared object) Linux में -> छोटी .exe, memory share.
- Step 5 Loader: .exe -> RAM -> CPU चलाता है. Runtime errors (divide by zero) यहाँ.
  - → Loader के 4 काम: Allocation (जगह देना), Linking (references जोड़ना), Relocation (addresses adjust), Loading (code memory में रखना).
  - → Relocation = program के हिस्सों को load address देना और code/data के addresses उसी हिसाब से बदलना (NET Dec 2013).
  - → Bootstrap loader: ROM (Read Only Memory) में रहता है, power on पर OS को RAM में लाता है.
- Turbo C++ shortcuts (BSEB (Bihar School Examination Board) lab): Alt+F9 = सिर्फ compile, Ctrl+F9 = compile + run, Alt+F5 = output screen देखना, F2 = save.
- File extensions: source = .cpp; header = .h. System header: #include <iostream>; user-defined header: #include "myfile.h".

```
hello.cpp --[preprocessor]--> expanded source
          --[compiler]-----> hello.s   (assembly)
          --[assembler]----> hello.o   (object code)
 hello.o + libraries --[linker]--> hello.exe
 hello.exe --[loader]--> RAM --> CPU runs it

// hello.cpp
#include <iostream>        // preprocessor pastes header
```

```
using namespace std;
int main() {
    cout << "Hello";        // cout is linked from library
    return 0;
}
```

④ 4. Exam Point + Trap

- Main memory में हमेशा रहने वाला system software (BPSC key) = Loader.
- Trap: Linker absolute को symbolic में नहीं बदलता; symbolic -> address करता है (NET Jan 2017).
- Trap: Alt+F9 सिर्फ compile; run के लिए Ctrl+F9.
- Trap: .cpp = source file, header नहीं. User-defined header = .h.

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- Type 1 — Shortcut key (TRE 1.0 Q50): 'compile and run in C++' -> Alt+F9 / Alt+F5 / Ctrl+F9. जवाब Ctrl+F9. Solve: Turbo C++ में खुद एक बार चलाकर याद करो: Alt = अकेला काम, Ctrl = पूरा काम.
- Type 2 — File extension (TRE 1.0 Q52): 'user-defined header file extension' -> hg / cpp / h. जवाब h.
- Type 3 — Which resides in main memory (TRE 2.0 Q90): Loader / Linker / Assembler. जवाब Loader. Logic: बाकी programs को memory में लाने के लिए loader (OS का resident हिस्सा) पहले से memory में चाहिए; linker/assembler ज़रूरत पर ही आते हैं.

⑥ 6. NET level में ऐसे आता है

- Statement type (NET Jan 2017): 'I. Linker कई object modules को एक load module में जोड़ता है II. Linker absolute references को symbolic से बदलता है' -> Only I.
- Definition type (NET Dec 2013): load addresses देना + code/data adjust करना = Relocation.
- Comparison: static vs dynamic linking (size, memory sharing).

7. याद रखो

- Preprocessor -> Compiler -> Assembler -> Linker -> Loader. (याद करने का वाक्य: 'P C A L L').
- Linker = डिब्बे जोड़ना; Loader = platform पर लाना. Ctrl+F9 = run, .h = header.

Compile vs Interpret — Worked Example (C++ और Python)

1. आसान भाषा में

- एक जैसा program, एक जैसी गलती: C++ (compiler) और Python (interpreter) अलग तरह से react करते हैं.
- Compiler में error हो तो एक भी line नहीं चलती. Interpreter में error से पहले वाली lines चल जाती हैं.

2. रोज़मर्रा का example

- Exam की copy:
  - → Teacher पूरी copy जाँचकर अंत में सारी गलतियाँ बताए, गलती है तो result (pass) नहीं = Compiler.
  - → Teacher हर जवाब के साथ नंबर देता जाए और पहली बड़ी गलती पर रुक जाए = Interpreter.

3. Example step by step

- नीचे दोनों programs में line 3 पर गलती है (C++: semicolon missing; Python: print की spelling गलत).
- C++: g++ पूरी file पढ़ता है -> error list देता है -> कोई .exe नहीं बनती -> output कुछ नहीं.
- Python: line 1 चली ('Start'), line 2 चली (30), line 3 पर NameError और रुक गया.

- Python अंदर से (SCERT/NCERT से आगे, NET level): CPython पहले source को bytecode (.pyc, __pycache__ folder) में compile करता है, फिर PVM (Python Virtual Machine) उसे interpret करती है. इसलिए 'interpreted language', पर अंदर compile + interpret दोनों.
- Java: javac -> .class bytecode -> JVM (Java Virtual Machine) interpret + JIT (Just-In-Time) compile. 'Write Once, Run Anywhere'.
- NCERT Ch 5: Python के 2 mode: Interactive mode (>>> prompt पर एक statement) और Script mode (.py file).

```
// C++  (sum.cpp)                    # Python (sum.py)
#include <iostream>                   print("Start")
using namespace std;                  a, b = 10, 20
int main() {                          print(a + b)
  cout << "Start" << endl;            prnt("End")    # NameError here
  int a = 10, b = 20     // ; missing
  cout << a + b << endl;
}

$ g++ sum.cpp   -> error: expected ';' ...  (no output at all)
$ python sum.py -> Start / 30 / NameError: name 'prnt' ...
```

④ 4. Exam Point + Trap

- Interpreter में error से पहले की lines का output आता है; compiler में error हो तो कुछ नहीं चलता.
- Trap: Java को सिर्फ 'compiled' या सिर्फ 'interpreted' मत कहो, यह hybrid है (bytecode + JVM).
- Trap: Python के bytecode (.pyc) बनने से वो compiled language नहीं बन जाती.

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- इस worked example जैसा सवाल TRE में नहीं आया, पर इसके concept से TRE 2.0 Q81 (statement-by-statement = interpreter) और TRE 1.0 Q44 (compiler: whole program, object code) हल होते हैं.

6. NET level में ऐसे आता है

- Statement type: 'I. Java bytecode platform independent है II. JVM bytecode को run time पर native code में बदल सकती है' -> दोनों सही.
- Output prediction: 'Error line 5 पर है, interpreter कितनी lines चलाएगा?' -> 4.

7. याद रखो

- Compiler = पूरी copy, फिर result. Interpreter = हर जवाब के साथ नंबर, पहली गलती पर stop.
- Java/Python = bytecode + virtual machine (hybrid).

Compiler के Phases (SCERT/NCERT से आगे, NET level)

1. आसान भाषा में

- (SCERT/NCERT से आगे, NET level) Compiler एक झटके में translate नहीं करता, वो 6 phases (चरणों) में काम करता है.
- Front end (analysis): source पढ़कर समझना: lexical, syntax, semantic, intermediate code. Language पर निर्भर, machine से स्वतंत्र.
- Back end (synthesis): target code बनाना: code optimisation, code generation. Machine पर निर्भर.
- Symbol table manager (नामों की list) और error handler सभी phases के साथ काम करते हैं.

2. रोज़मर्रा का example

- एक हिंदी वाक्य को अंग्रेज़ी में बदलना:
  → वाक्य को शब्दों में तोड़ो = Lexical (tokens).
  → Grammar सही है? (कर्ता, क्रिया) = Syntax (parse tree).

→ मतलब सही है? 'पत्थर खाना खाता है' grammar सही पर मतलब गलत = Semantic.

→ कच्चा अनुवाद = Intermediate code; छोटा/सुंदर करो = Optimisation; अंतिम साफ़ copy = Code generation.

③ 3. हर phase step by step

- 1. Lexical analysis (scanner): characters -> tokens (keyword, identifier, constant, operator, punctuation). Comments और white space हटाता है. Regular expressions और finite automata, DFA (Deterministic Finite Automaton) पर आधारित. Lexeme = token की असली string (जैसे 'rate').
- 2. Syntax analysis (parser): tokens grammar के हिसाब से हैं? CFG (Context Free Grammar) से parse tree. Missing semicolon, unbalanced bracket यहाँ पकड़े जाते हैं.
- 3. Semantic analysis: मतलब check: type checking, undeclared variable, type conversion (int -> float).
- 4. Intermediate code generation: machine-independent code, जैसे TAC (Three-Address Code): हर instruction में ज़्यादा से ज़्यादा 3 addresses.
- 5. Code optimisation: बिना मतलब बदले code छोटा/तेज़: constant folding, common sub-expression elimination, dead code elimination, loop optimisation.
- 6. Code generation: target assembly/machine code, register allocation.
- Classic example: position = initial + rate 60. Tokens: <id,1> <=> <id,2> <+> <id,3> <> <60>. Semantic phase rate (float) 60 (int) देखकर inttofloat(60) डालता है. Optimiser 60 को compile time पर ही 60.0 बना देता है.
- Pass vs Phase: Phase = logical step; Pass = source (या intermediate रूप) को एक बार पूरा पढ़ना. कई phases एक pass में हो

सकते हैं.

→ One-pass compiler: तेज़, कम memory, optimisation कम, forward reference मुश्किल. इसलिए Pascal में 'declare before use' rule है.

→ Multi-pass compiler: ज़्यादा memory/समय, पर बेहतर optimisation और forward references आसान.

```
Source program (characters)
   -> 1 Lexical Analyzer      -> tokens
   -> 2 Syntax Analyzer       -> parse tree
   -> 3 Semantic Analyzer     -> annotated tree
   -> 4 Intermediate Code Gen -> three-address code
   -> 5 Code Optimizer        -> better intermediate code
   -> 6 Code Generator        -> target machine code
   [ Symbol Table + Error Handler talk to all phases ]

position = initial + rate * 60
 TAC:  t1 = inttofloat(60)      Optimised:
       t2 = id3 * t1             t1 = id3 * 60.0
       t3 = id2 + t2             id1 = id2 + t1
       id1 = t3
 Code: LDF R2, id3 ; MULF R2, R2, #60.0
       LDF R1, id2 ; ADDF R1, R1, R2 ; STF id1, R1
```

④ 4. Exam Point + Trap

- Tokens = lexical; parse tree = syntax; type checking = semantic; register allocation = code generation.
- Trap: missing semicolon syntax error है, lexical नहीं. Undeclared variable semantic error है.
- Trap: Front end machine-independent, back end machine-dependent (उल्टा नहीं).

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- Compiler phases पर BPSC TRE 1.0 / 2.0 / 3.0 में अब तक कोई सवाल नहीं आया (TRE 3.0 में NET जैसे numericals OS (Operating System) से आए). TRE 4.0 के लिए NET level तैयारी रखो.

6. NET level में ऐसे आता है

- Phase पहचानो: 'Which phase groups characters into tokens?' -> Lexical; 'type checking?' -> Semantic.
- Token counting (numerical): printf("i = %d", i); -> printf ("..." , i) ; = 7 tokens. String पूरी एक token, comment और space token नहीं (Numericals chapter देखो).
- Match type: lexical-DFA, syntax-CFG, intermediate-TAC, optimisation-constant folding.

7. याद रखो

- क्रम: Lexical -> Syntax -> Semantic -> Intermediate -> Optimise -> Code gen. (याद करो: 'Lala Sabzi Se Itna Order Close').
- Symbol table + error handler सब phases के साथ.

Assembler, Loader के प्रकार और Macro (SCERT/NCERT से आगे, NET level)

1. आसान भाषा में

- (SCERT/NCERT से आगे, NET level) Assembly program में label (जैसे JMP LOOP) कभी पहले use होता है और बाद में define, इसे forward reference कहते हैं. इसे हल करने के लिए assembler दो बार program पढ़ता है (two-pass).
- Loader कई तरह के होते हैं: absolute, relocating, direct linking.
- Macro = instructions के group को एक नाम देना; जहाँ नाम लिखो वहाँ पूरी body चिपक (expand) जाती है.

2. रोज़मर्रा का example

- Two-pass = किताब पढ़ना: पहली बार पढ़कर index (symbol table) बनाओ कि कौन सा chapter किस page पर; दूसरी बार पढ़ते हुए 'page X

देखो वाली जगहों पर page number भर दो.

- Macro = WhatsApp का quick reply: छोटा shortcut लिखो, पूरा message अपने-आप भर जाता है (message लंबा, पर typing का समय बचा).

③ 3. हर प्रकार step by step

- Pass I: LC (Location Counter) बढ़ाते हुए हर label का address -> symbol table (और literal table). Pseudo-ops (START, END, ORG, EQU, DS, DC) process.
- Pass 2: MOT (Machine Opcode Table) / OPTAB और symbol table से असली machine code + object program. POT (Pseudo Opcode Table) = assembler directives की table.
- One-pass assembler: forward references को backpatching से भरता है (address पता चलते ही पीछे जाकर भरो).
- Load-and-go assembler: object code सीधे memory में रखकर तुरंत चलाता है, disk पर object file लिखने का overhead नहीं.
- Compile-and-go (assemble-and-go) loader: translator ही program memory में रखकर चलाता है; हर बार दोबारा translate, memory waste.
- Absolute loader: object code में लिखे fixed address पर load. न relocation, न linking. Simple पर inflexible.
- Relocating loader: program को किसी भी खाली जगह load; relocation bits या modification records से addresses adjust.
- Direct linking loader: linking + relocation + loading तीनों; EXTDEF (External Definition) / EXTREF (External Reference) resolve करता है.
- Dynamic loading: routine तभी load जब call हो (memory बचत). Dynamic linking: run time पर library जोड़ना.
- Macro processor: MACRO ... MEND के बीच body. हर macro call की जगह body expand. Two-pass macro processor की tables: MNT (Macro Name Table), MDT (Macro Definition Table), ALA

(Argument List Array); MNTC/MDTC (MNT Counter / MDT Counter).

- Macro vs subroutine: macro -> program बड़ा, CALL/RET overhead नहीं, expansion translation time पर. Subroutine -> एक copy, पर run time पर call/return.
- C++ में macro = #define (preprocessor). Bracket ज़रूरी है, नहीं तो गलत जवाब:

```
#define SQR(x) ((x)*(x))
int y = SQR(5);      // becomes: int y = ((5)*(5));  -> 25

#define SQ(x) x*x    // no brackets
int z = SQ(2+3);     // becomes: 2+3*2+3 = 2+6+3 = 11 (not 25)
```

4. 4. Exam Point + Trap

- Two-pass assembler में symbol table first pass में बनती है (NET Dec 2014).
- 'No relocation' = absolute loader; 'relocation bits' = relocating loader.
- 'Macro expansion increases code size but saves call overhead'.
- Trap: macro expansion loader run time पर नहीं करता; preprocessor/assembler translation time पर करता है.

5. 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- Assembler की basic definition (assembly -> machine = assembler) TRE 2.0 Q79 में आई.
- Two-pass, loader के प्रकार और macro processor पर TRE 1.0 / 2.0 / 3.0 में कोई सवाल नहीं आया.

6. 6. NET level में ऐसे आता है

- Which pass type: symbol table first pass, code generation second pass.
- Definition: 'object program को assembler के दिए address पर, बिना

relocation' -> absolute loader.

- Comparison: macro vs subroutine (size बनाम speed).
- Numerical: LC trace करके symbol table भरना (Numericals chapter देखो).

7 7. याद रखो

- Pass 1 = index बनाना (symbol table); Pass 2 = page number भरना (code).
- Absolute = fixed जगह; Relocating = कहीं भी; Direct linking = तीनों काम. Macro = quick reply.

Bootstrapping, Cross-compiler, Bytecode, JIT (SCERT/NCERT से आगे, NET level)

1 1. आसान भाषा में

- (SCERT/NCERT से आगे, NET level) Bootstrapping: किसी language का compiler उसी language में लिखना. शुरुआत छोटे subset compiler से (assembly या दूसरी language में), फिर उससे बड़ा compiler compile -> self-hosting compiler.
- Cross compiler: एक machine पर चलकर दूसरी machine का code बनाए.
- JIT (Just-In-Time) compiler: program चलते समय bytecode को native machine code में बदलता है.

2 2. रोज़मर्रा का example

→Bootstrapping = साइकिल चलाना सीखना: पहले छोटी साइकिल (subset), उसी से बड़ी साइकिल चलाना सीखो.

→Cross compiler = पटना की bakery में दिल्ली की shop के लिए केक

बनाना (बनाया यहाँ, चलेगा वहाँ).

→JIT = हर दिन आने वाले ग्राहक का order पहले से याद कर लेना (hot code cache) ताकि अगली बार जल्दी हो.

③ 3. हर concept step by step

- T-diagram: compiler के 3 हिस्से: Source language S, Target language T, Implementation language I. जैसे C compiler जो C में लिखा है और x86 code देता है.
- Bootstrap loader (boot time, ROM) और compiler bootstrapping: नाम मिलता-जुलता, concept अलग.
- Cross compiler: PC (Personal Computer) पर Arduino / ARM (Advanced RISC Machines) के लिए code. RISC = Reduced Instruction Set Computer. Embedded systems में common.
- Decompiler: machine/bytecode -> HLL (High-Level Language) जैसा code (reverse engineering). Disassembler: machine code -> assembly.
- Transpiler (source-to-source compiler): एक HLL -> दूसरी HLL (TypeScript -> JavaScript).
- Bytecode: platform-independent intermediate code (Java .class, Python .pyc) जिसे virtual machine चलाती है.
- JIT: बार-बार चलने वाले (hot) bytecode को native code में बदलकर cache -> interpreter से तेज़. Java HotSpot JVM (Java Virtual Machine), .NET CLR (Common Language Runtime), PyPy.
- AOT (Ahead-Of-Time) compilation: चलाने से पहले ही native code: C/C++ compilers.
- Hybrid: Java / C# = compile (bytecode) + interpret/JIT.

④ 4. Exam Point + Trap

- 'Runs on A, produces code for B' = cross compiler. 'Compiler written in its own language' = bootstrapping.
- Trap: JIT run time पर compile करता है; C++ compiler AOT है.

- Trap: decompiler (machine -> HLL) और disassembler (machine -> assembly) अलग.

⑤ 5. BPSC ऐसे पूछता है (TRE 1.0 / 2.0 / 3.0)

- इस sub-topic पर BPSC TRE 1.0 / 2.0 / 3.0 में कोई सवाल नहीं आया. सिर्फ NET level के लिए.

⑥ 6. NET level में ऐसे आता है

- Definition type: cross compiler, bootstrapping, JIT (Practice questions देखो).
- Statement type: 'I. javac native machine code बनाता है II. JVM bytecode चलाती है' -> I गलत (bytecode बनाता है), II सही.

⑦ 7. याद रखो

- Bootstrapping = छोटी साइकिल से बड़ी; Cross = यहाँ बनाओ, वहाँ चलाओ; JIT = चलते-चलते compile; AOT = पहले से compile.

Numericals / Practice — हल करके सीखो

① Practice 1: Token गिनना (C++)

- Rule: keyword, identifier, constant, operator, punctuation (; , ( ) { }) = एक-एक token. पूरी string "..." = 1 token. Space और comment = 0 token. '>=' और '<<' दो characters पर 1 token.

```
(a) int sum = a + 10;
    int | sum | = | a | + | 10 | ;          -> 7 tokens
(b) if (x >= 10) y = x * 2;
    if ( x >= 10 ) y = x * 2 ;             -> 12 tokens
(c) cout << "Hello World";   // greet
    cout | << | "Hello World" | ;     -> 4 tokens (no comment)
(d) printf("i = %d, &i = %x", i, &i);
    printf ( "..." , i , & i ) ;           -> 10 tokens
```

② Practice 2: Compiler phases trace

- Statement: total = price + tax 2 (price, tax float; 2 int). हर phase का output लिखो.

```
Lexical : <id,1> <=> <id,2> <+> <id,3> <*> <2>      (7 tokens)
Syntax  : tree  = ( id1 , + ( id2 , * ( id3 , 2 ) ) )
Semantic: tax is float, 2 is int -> inttofloat(2)
TAC     : t1 = inttofloat(2)
          t2 = id3 * t1
          t3 = id2 + t2
          id1 = t3
Optimise: t1 = id3 * 2.0
          id1 = id2 + t1
Code    : LDF R2, id3 ; MULF R2, R2, #2.0
          LDF R1, id2 ; ADDF R1, R1, R2 ; STF id1, R1
```

③ Practice 3: Two-pass assembler — LC और Symbol table

- मान लो हर instruction 1 word लेता है, START 100. Pass 1 में LC (Location Counter) लिखकर symbol table बनाओ.

```
        START 100
LOOP    MOVER AREG, X     LC = 100
        ADD   AREG, ONE   LC = 101
        BC    ANY, LOOP   LC = 102
X       DS    1           LC = 103
ONE     DC    '1'         LC = 104
        END

Symbol table (Pass 1):  LOOP = 100,  X = 103,  ONE = 104
Pass 2: X and ONE are forward references -> filled from table
```

④ Practice 4: Macro expansion (C++ #define)

- Bracket वाला और बिना bracket वाला macro: जवाब अलग.

```
#define SQ(x)  x*x
#define SQR(x) ((x)*(x))
SQ(2+3)   -> 2+3*2+3   = 2+6+3 = 11
SQR(2+3)  -> ((2+3)*(2+3)) = 5*5 = 25
```

```
SQ(4)     -> 4*4 = 16          (same when no operator)

Size: macro body 5 lines, called 3 times
  macro      -> 5 x 3 = 15 lines (no CALL/RET)
  subroutine -> 5 lines + 3 CALL = 8 lines (call overhead)
```

⑤ Practice 5: Compiler vs Interpreter — कितनी lines चलेंगी?

- 100 lines का program, line 40 पर syntax error. Output कितनी lines का?

```
Interpreter : lines 1..39 run, stops at 40  -> 39 lines run
Compiler    : error before running, no .exe -> 0 lines run
After fixing: compiler translates once, runs many times
              interpreter translates every run
```

⑥ Practice 6: Match करो (licence + generation)

- बाएँ को दाएँ से मिलाओ, फिर जवाब देखो.

```
(i)   Shareware      (a) copyleft
(ii)  GPL            (b) trial then pay
(iii) Freeware       (c) free price, closed source
(iv)  SQL            (d) 4GL
(v)   Prolog         (e) 5GL
Answer: i-b, ii-a, iii-c, iv-d, v-e
```

Abbreviations — Full Forms

① Software & Language Translators (A–Z)

- AI — Artificial Intelligence
- ALA — Argument List Array
- AOT — Ahead-Of-Time
- ARM — Advanced RISC Machines
- BASIC — Beginner's All-purpose Symbolic Instruction Code
- BIOS — Basic Input Output System

- BPSC — Bihar Public Service Commission
- BSD — Berkeley Software Distribution
- BSEB — Bihar School Examination Board
- CC — Creative Commons
- CFG — Context Free Grammar
- CLR — Common Language Runtime
- COBOL — COmmon Business Oriented Language
- CPU — Central Processing Unit
- DFA — Deterministic Finite Automaton
- DLL — Dynamic Link Library
- EULA — End User License Agreement
- EXTDEF / EXTREF — External Definition / External Reference
- FORTRAN — FORmula TRANslation
- FOSS — Free and Open Source Software
- FSF — Free Software Foundation
- GL — Generation Language (IGL-SGL)
- GNU — GNU's Not Unix
- GPL — General Public License
- HLL — High-Level Language
- IBM — International Business Machines
- IDE — Integrated Development Environment
- IDLE — Integrated Development and Learning Environment
- I/O — Input/Output
- IPR — Intellectual Property Right
- JIT — Just-In-Time
- JVM — Java Virtual Machine
- LC — Location Counter
- MATLAB — MATrix LABoratory
- MDT — Macro Definition Table
- MIT — Massachusetts Institute of Technology

- MNT — Macro Name Table
- MOT — Machine Opcode Table
- NCERT — National Council of Educational Research and Training
- NET — National Eligibility Test
- OS — Operating System
- PC — Personal Computer
- POST — Power-On Self-Test
- POT — Pseudo Opcode Table
- Prolog — PROgramming in LOGic
- PVM — Python Virtual Machine
- RAM — Random Access Memory
- RISC — Reduced Instruction Set Computer
- ROM — Read Only Memory
- SCERT — State Council of Educational Research and Training
- SQL — Structured Query Language
- TAC — Three-Address Code
- TRE — Teacher Recruitment Exam
- UEFI — Unified Extensible Firmware Interface
- UGC — University Grants Commission

(2) File extensions

- .cpp — C++ source file
- .h — header file
- .obj / .o — object file
- .exe — executable
- .so — shared object (Linux)
- .dll — dynamic link library (Windows)
- .class — Java bytecode
- .pyc — Python bytecode

Questions + हर सवाल के नीचे Explanation

Q1. [BPSC TRE 1.0 (2023) Q42]

Which language is better for writing structured code?

(A) PASCAL
(B) FORTRAN
(C) BASIC
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: Pascal को Niklaus Wirth ने structured programming सिखाने के लिए बनाया
(B) FORTRAN scientific गणित के लिए बनी, structure पर ज़ोर नहीं (पुराने version में GOTO बहुत)
(C) BASIC शुरुआती लोगों के लिए आसान language, structured नहीं थी
(D) सिर्फ A सही है
(E) A सही है, इसलिए गलत

Ans: (A) PASCAL

Pascal में block structure (begin...end), if / while / repeat जैसे control structures हैं, इसलिए structured code के लिए यह सबसे अच्छी मानी गई. BPSC key: (A).

Q2. [BPSC TRE 1.0 (2023) Q44]

Select the true statement for compiler.

(A) The input of the compiler is source program.
(B) It translates the source code into object code as a whole.
(C) The output of the compiler is object code.
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: compiler का input source program (HLL (High-Level Language) code) होता है
(B) ✔ सही: compiler पूरे program को एक साथ object code में बदलता है
(C) ✔ सही: compiler का output object code होता है
(D) ✔ सही जवाब: A, B, C तीनों सही हैं
(E) गलत: ऊपर के options सही हैं

Ans: (D) More than one of the above

Compiler: input = source program, पूरा program एक साथ translate, output = object code. तीनों statement सही, इसलिए (D) More than one of the above.

Q3. [BPSC TRE 1.0 (2023) Q50]

Which shortcut key is used to compile and run the program in C++?

(A) Alt+F9
(B) Alt+F5
(C) Ctrl+F9
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) Alt+F9 सिर्फ compile करता है, run नहीं
(B) Alt+F5 सिर्फ output (user) screen दिखाता है
(C) ✔ सही: Ctrl+F9 = compile + run (Turbo C++)
(D) सिर्फ C सही है
(E) C सही है, इसलिए गलत

Ans: (C) Ctrl+F9

Turbo C++ IDE (Integrated Development Environment) में Ctrl+F9 दबाने से program compile भी होता है और run भी. Alt+F9 = सिर्फ compile, Alt+F5 = output screen.

Q4. [BPSC TRE 1.0 (2023) Q52]

Which of the following is userdefined header file extension used in C++?

(A) hg
(B) cpp
(C) h
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) '.hg' C++ की कोई header extension नहीं
(B) .cpp C++ की source file की extension है, header की नहीं
(C) ✔ सही: user-defined header file .h होती है, जैसे #include "myfile.h"
(D) सिर्फ C सही है
(E) C सही है, इसलिए गलत

Ans: (C) h

C++ में अपनी header file .h extension से बनाते हैं और #include "myfile.h" (double quotes) से जोड़ते हैं. System header जैसे <iostream> angle brackets में.

Q5. [BPSC TRE 2.0 (2023) Q79]

A computer program that converts an assembly language program into equivalent machine language program is called

(A) compiler
(B) assembler
(C) linker
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) Compiler HLL को machine language में बदलता है, assembly को नहीं
(B) ✔ सही: assembler assembly (mnemonics) को machine language में बदलता है
(C) Linker object files और libraries जोड़ता है, translate नहीं करता
(D) सिर्फ B सही है
(E) B सही है, इसलिए गलत

Ans: (B) assembler

Assembly language (MOV, ADD जैसे mnemonics) -> machine code (0/1) = assembler. BPSC key (provisional): (B).

Q6. [BPSC TRE 2.0 (2023) Q80]

A computer program that converts a high-level language program into an equivalent machine language program is called

(A) assembler
(B) compiler
(C) loader
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) Assembler assembly language के लिए है, HLL के लिए नहीं
(B) ✔ सही: compiler पूरे HLL program को equivalent machine language program में बदलता है
(C) Loader program को memory में लाता है, translate नहीं करता
(D) सिर्फ B सही है
(E) B सही है, इसलिए गलत

Ans: (B) compiler

High-level language (C++) का पूरा program -> उसके बराबर machine language program = compiler. BPSC key (provisional): (B).

Q7. [BPSC TRE 2.0 (2023) Q81]

A computer program that translates high-level language program into machine language program statement-by-statement is called

(A) compiler
(B) loader
(C) interpreter
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) Compiler पूरा program एक साथ translate करता है, statement-by-statement नहीं
(B) Loader translate नहीं करता, memory में लाता है
(C) ✔ सही: interpreter एक-एक statement translate करके तुरंत चलाता है
(D) सिर्फ C सही है
(E) C सही है, इसलिए गलत

Ans: (C) interpreter

'Statement-by-statement' keyword = interpreter (जैसे Python). Object file नहीं बनती, हर run में interpreter चाहिए. BPSC key (provisional): (C).

Q8. [BPSC TRE 2.0 (2023) Q84]

Which of the following is not a system tool?

(A) Scandisk
(B) Format
(C) Folder
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) Scandisk disk की गलतियाँ जाँचने वाला system tool है
(B) Format disk तैयार करने वाला system tool है
(C) ✔ सही: Folder सिर्फ files रखने का container है, tool नहीं
(D) सिर्फ C 'not a tool' है
(E) C सही है, इसलिए गलत

Ans: (C) Folder

Scandisk और Format system utilities हैं जो disk पर काम करते हैं. Folder कोई काम नहीं करता, बस files रखता है. BPSC key (provisional): (C).

Q9. [BPSC TRE 2.0 (2023) Q90]

Which of the following system softwares resides always in main memory?

(A) Loader
(B) Linker
(C) Assembler
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) ✔ सही: loader (OS (Operating System) का resident हिस्सा) memory में रहता है ताकि दूसरे programs load कर सके
(B) Linker ज़रूरत पर ही चलता है, हमेशा memory में नहीं रहता
(C) Assembler translation के समय ही memory में आता है
(D) सिर्फ A सही है
(E) A सही है, इसलिए गलत

Ans: (A) Loader

बाकी हर program को main memory में लाने का काम loader का है, इसलिए वह पहले से memory में होना चाहिए. Linker और assembler ज़रूरत पर load होते हैं. BPSC key (provisional): (A). (Bootstrap loader ROM में होता है, वो अलग बात है.)

Q10. [BPSC TRE 2.0 (2023) Q107]

Group of instructions that directs a computer is called

(A) memory
(B) storage
(C) program
(D) More than one of the above
(E) None of the above

हर **option** समझो:

(A) Memory सिर्फ data और program रखती है, निर्देश नहीं देती
(B) Storage भी सिर्फ रखने की जगह है
(C) ✔ सही: computer को निर्देश देने वाले instructions का group = program
(D) सिर्फ C सही है
(E) C सही है, इसलिए गलत

Ans: (C) program

Program = किसी काम के लिए instructions का क्रम से लिखा group. Memory/storage उसे सिर्फ रखते हैं. BPSC key (provisional): (C).

Q11. [UGC NET Dec 2013]

The process of assigning load addresses to the various parts of the program and adjusting the code and data in the program to reflect the assigned addresses is called _____.

(A) Symbol resolution
(B) Parsing
(C) Assembly
(D) Relocation

हर **option** समझो:

(A) Symbol resolution = external नामों को उनके address से जोड़ना, address adjust करना नहीं
(B) Parsing = syntax analysis (grammar check), compiler का phase
(C) Assembly = assembly language को machine code में बदलना
(D) ✔ सही: load address देना + code/data के addresses adjust करना = Relocation

Ans: (D) Relocation

Program को memory में किसी भी जगह रखो, तो उसके अंदर के addresses उसी हिसाब से बदलने पड़ते हैं. यही Relocation है (linker / relocating loader का काम).

Q12. [UGC NET Dec 2014]

In a two-pass assembler, symbol table is

(A) Generated in first pass
(B) Generated in second pass
(C) Not generated at all
(D) Generated and used only in second pass

हर **option** समझो:

(A) ✔ सही: Pass 1 में LC (Location Counter) से हर label का address निकालकर symbol table बनती है
(B) Pass 2 symbol table को use करता है, बनाता नहीं
(C) Symbol table बनती ही है, तभी forward reference हल होते हैं
(D) Second pass में सिर्फ use होती है, generate पहले pass में

Ans: (A) Generated in first pass

Two-pass assembler: Pass 1 = symbol table बनाना (index बनाना), Pass 2 = उसी table से machine code बनाना (page number भरना).

Q13. [UGC NET Jan 2017]

Which of the following statement(s) regarding a linker software is/are true ? I. A function of a linker is to combine several object modules into a single load module. II A function of a linker is to replace absolute references in an object module by symbolic references to locations in other modules.

(A) Only I
(B) Only II
(C) Both I and II
(D) Neither I nor II

हर **option** समझो:

(A) ✔ सही: Statement I सही, II उल्टा है, इसलिए Only I
(B) Statement II गलत है: linker symbolic को address में बदलता है, उल्टा नहीं
(C) दोनों सही नहीं, II गलत है
(D) Statement I सही है, इसलिए 'Neither' गलत

Ans: (A) Only I

I: linker कई object modules को एक load module में जोड़ता है, सही. II: linker symbolic references को addresses से replace करता है, absolute को symbolic से नहीं, इसलिए II गलत. जवाब Only I.

Q14. [UGC NET Dec 2022]

Machine Level Language is a/an

(A) Assembly Language
(B) Low level language
(C) High level Language
(D) Translating Language

हर **option** समझो:

(A) Assembly language भी low level है, पर machine language assembly नहीं है
(B) ✔ सही: machine language (0/1) = low level language (1GL (First Generation Language))
(C) High level language English जैसी होती है, machine language नहीं
(D) 'Translating language' कोई category नहीं, translator एक program होता है

Ans: (B) Low level language

Machine language सिर्फ 0 और 1 में होती है जिसे CPU (Central Processing Unit) सीधे समझता है. यह low-level language है.

Q15. [Practice]

Which of the following is NOT an example of system software?

(A) Device driver
(B) Disk defragmenter
(C) Operating system
(D) Spreadsheet program

हर **option** समझो:

(A) Device driver OS और device के बीच mediator, system software है
(B) Disk defragmenter system utility है, system software का हिस्सा
(C) Operating system सबसे basic system software है
(D) ✔ सही: Spreadsheet (LibreOffice Calc) user का काम करता है = application software

Ans: (D) Spreadsheet program

OS (Operating System), device drivers और utilities = system software.
Spreadsheet हिसाब-किताब जैसा user का काम करता है, इसलिए application software.

Q16. [Practice]

Software that is permanently stored in ROM/flash and initialises the hardware when a computer is switched on is called:

(A) Firmware
(B) Freeware
(C) Shareware
(D) Middleware

हर **option** समझो:

(A) ✔ सही: ROM (Read Only Memory) / flash में हमेशा रहने वाला software = firmware (BIOS (Basic Input Output System))
(B) Freeware licence का प्रकार है (free use), storage से मतलब नहीं
(C) Shareware भी licence का प्रकार है (trial then pay)
(D) Middleware OS और applications के बीच glue software है

Ans: (A) Firmware

Power on पर hardware को शुरू (initialise) करने वाला BIOS/UEFI (Unified Extensible Firmware Interface) ROM/flash में रहता है, यही firmware है.

Q17. [Practice]

Software that can be used free for a trial period, after which the user is expected to pay to continue using it, is:

(A) Freeware
(B) Shareware
(C) Open source software
(D) Public domain software

हर **option** समझो:

(A) Freeware हमेशा free होता है, trial के बाद पैसे नहीं माँगता
(B) ✔ सही: Shareware = trial free, बाद में पैसे ('try before you buy')
(C) Open source में source code भी मिलता है, trial-payment वाली शर्त नहीं
(D) Public domain पर कोई copyright ही नहीं, पैसे का सवाल नहीं

Ans: (B) Shareware

Shareware कुछ दिन मुफ़्त चलता है, फिर license खरीदना पड़ता है (जैसे WinRAR trial).

Q18. [Practice]

Under the GNU GPL (General Public License), which condition applies when a modified version of the software is distributed?

(A) Its source code must be kept secret
(B) It must also be released under the GPL with source code available
(C) It can never be sold for a fee
(D) It may be used only for non-commercial purposes

हर **option** समझो:

(A) उल्टा: GPL (General Public License) में source code देना ज़रूरी है
(B) ✔ सही: copyleft, modified version भी GPL में और source के साथ
(C) NCERT: GPL software की copies के लिए fee ले सकते हैं
(D) GPL commercial use भी allow करता है

Ans: (B) It must also be released under the GPL with source code available

GPL copyleft licence है: बदला version बाँटो तो वो भी GPL में और source code के साथ. Fee लेना allowed है, इसलिए (C) गलत.

Q19. [Practice]

Assembly language belongs to which generation of programming languages?

(A) First generation (1GL)
(B) Second generation (2GL)
(C) Third generation (3GL)
(D) Fourth generation (4GL)

हर **option** समझो:

(A) 1GL (First Generation Language) = machine language (0/1)
(B) ✔ सही: 2GL = assembly language (MOV, ADD जैसे mnemonics)
(C) 3GL = C, C++, Java जैसी HLL (High-Level Language)
(D) 4GL = SQL जैसी non-procedural language

Ans: (B) Second generation (2GL)

1GL machine, 2GL assembly, 3GL C++/Java/Python, 4GL SQL, 5GL Prolog.

Q20. [Practice]

SQL (Structured Query Language) is generally classified as a:

(A) Second generation language
(B) Third generation language
(C) Fourth generation language
(D) Fifth generation language

हर **option** समझो:

(A) 2GL assembly language है
(B) 3GL procedural है (कैसे करना है, step by step लिखते हैं)
(C) ✔ सही: SQL (Structured Query Language) non-procedural 4GL है
(D) 5GL logic/AI (Artificial Intelligence) वाली languages हैं, जैसे Prolog

Ans: (C) Fourth generation language

SQL में हम सिर्फ बताते हैं क्या चाहिए (SELECT ... WHERE), कैसे निकालना है वो DBMS (Database Management System) तय करता है. इसलिए 4GL.

Q21. [Practice]

Prolog, a logic programming language used in AI (Artificial Intelligence), is usually placed in the:

(A) Third generation
(B) Fourth generation
(C) Fifth generation
(D) First generation

हर **option** समझो:

(A) 3GL procedural languages हैं (C, C++)
(B) 4GL SQL जैसी query languages हैं
(C) ✔ सही: Prolog (PROgramming in LOGic) = 5GL, logic से problem solve
(D) 1GL machine language है

Ans: (C) Fifth generation

5GL में rules/constraints देते हैं और system खुद हल ढूँढता है. Prolog AI और expert systems में use होती है.

Q22. [Practice]

Which phase of a compiler reads the source program character by character and groups the characters into tokens?

(A) Lexical analysis
(B) Syntax analysis
(C) Semantic analysis
(D) Code optimisation

हर **option** समझो:

(A) ✔ सही: lexical analyzer (scanner) characters को tokens में बाँटता है
(B) Syntax analysis tokens से parse tree बनाता है, characters नहीं पढ़ता
(C) Semantic analysis meaning / type check करता है
(D) Code optimisation code को तेज़/छोटा करता है

Ans: (A) Lexical analysis

Compiler का पहला phase lexical analysis है: source को character by character पढ़कर keyword, identifier, operator जैसे tokens बनाता है, comments/space हटाता है.

Q23. [Practice]

Type checking (for example, detecting that an integer is added to a string) is mainly performed in which compiler phase?

(A) Lexical analysis
(B) Syntax analysis
(C) Semantic analysis
(D) Code generation

हर **option** समझो:

(A) Lexical analysis सिर्फ tokens बनाता है, type नहीं देखता
(B) Syntax analysis सिर्फ grammar check करता है
(C) ✔ सही: type checking semantic analysis में होती है
(D) Code generation आखिरी phase है, machine code बनाता है

Ans: (C) Semantic analysis

Integer + string जोड़ना grammar में ठीक लग सकता है, पर मतलब गलत है. मतलब (type) check

= semantic analysis.

Q24. [Practice]

A compiler that runs on one machine (platform A) but produces executable code for a different machine (platform B) is called a:

(A) Bootstrap compiler
(B) Just-In-Time compiler
(C) Decompiler
(D) Cross compiler

हर **option** समझो:

(A) Bootstrap compiler = अपनी ही language में लिखा compiler
(B) JIT (Just-In-Time) run time पर उसी machine के लिए code बनाता है
(C) Decompiler machine code को HLL जैसा वापस बनाता है
(D) ✔ सही: एक platform पर चलकर दूसरे platform का code = cross compiler

Ans: (D) Cross compiler

जैसे PC (Personal Computer, x86) पर बैठकर ARM (Advanced RISC Machines) / Arduino के लिए code बनाना. Embedded systems में बहुत common.

Q25. [Practice]

In Java, the javac compiler translates a .java source file into:

(A) Native machine code of the CPU
(B) Bytecode in a .class file, executed by the JVM
(C) Assembly language
(D) A C++ source file

हर **option** समझो:

(A) javac native machine code नहीं बनाता, वो JVM (Java Virtual Machine) / JIT का काम है
(B) ✔ सही: javac .java को .class bytecode में बदलता है, JVM उसे चलाती है
(C) javac assembly language नहीं बनाता
(D) javac C++ code नहीं बनाता

Ans: (B) Bytecode in a .class file, executed by the JVM

Java: source (.java) -> javac -> bytecode (.class) -> JVM. Bytecode platform-independent है: 'write once, run anywhere'.

Q26. [Practice]

A JIT (Just-In-Time) compiler:

(A) Translates bytecode/intermediate code into native machine code at run time
(B) Compiles the whole source program before the software is distributed
(C) Executes source code line by line without any translation
(D) Converts machine code back into high-level source code

हर **option** समझो:

(A) ✔ सही: JIT run time पर bytecode को native machine code में बदलता है
(B) चलाने से पहले पूरा compile करना AOT (Ahead-Of-Time) है, JIT नहीं
(C) बिना translation line-by-line चलाना interpreter जैसा है, JIT नहीं
(D) Machine code से HLL बनाना decompiler का काम है

Ans: (A) Translates bytecode/intermediate code into native machine code at run time

JIT बार-बार चलने वाले (hot) bytecode को native code में बदलकर cache करता है, इसलिए interpreter से तेज़. Example: Java HotSpot, .NET CLR (Common Language Runtime).

Q27. [Practice]

Writing a compiler for a language in that same language and then compiling it with a simpler, already available version of the compiler is called:

(A) Cross compiling
(B) Linking
(C) Bootstrapping
(D) Relocation

हर **option** समझो:

(A) Cross compiling = दूसरे platform के लिए code बनाना
(B) Linking = object files जोड़ना
(C) ✔ सही: compiler को उसी language में लिखकर छोटे compiler से compile करना = bootstrapping
(D) Relocation = addresses adjust करना

Ans: (C) Bootstrapping

पहले छोटा subset compiler (assembly या दूसरी language में), फिर उससे पूरी language में लिखा बड़ा compiler compile. जैसे C compiler C में लिखा होना.

Q28. [Practice]

Which statement about an absolute loader is correct?

(A) It performs both relocation and linking
(B) It loads the object program at the address specified by the assembler/programmer, without relocation
(C) It loads routines only when they are called at run time
(D) It expands macros before loading

हर **option** समझो:

(A) Relocation + linking दोनों direct linking loader करता है, absolute नहीं
(B) ✔ सही: absolute loader assembler/programmer के दिए fixed address पर, बिना relocation load करता है
(C) Call होने पर load करना dynamic loading है
(D) Macro expansion preprocessor/macro processor करता है, loader नहीं

Ans: (B) It loads the object program at the address specified by the assembler/programmer, without relocation

Absolute loader सबसे simple है: object code वहीं रखता है जहाँ लिखा है. Relocation/linking नहीं, इसलिए program दूसरी जगह नहीं चल सकता.

Q29. [Practice]

Compared with a subroutine (function) call, using a macro generally:

(A) Increases program size but avoids the call/return overhead
(B) Reduces program size and adds call/return overhead
(C) Is executed with CALL and RET instructions
(D) Is expanded by the loader at run time

हर **option** समझो:

(A) ✔ सही: macro expansion से code बड़ा होता है, पर CALL/RET का overhead नहीं
(B) उल्टा: subroutine size कम करता है और call overhead जोड़ता है
(C) CALL और RET subroutine में होते हैं, macro में नहीं
(D) Macro translation time पर expand होता है, loader run time पर नहीं करता

Ans: (A) Increases program size but avoids the call/return overhead

Macro = हर call की जगह पूरी body चिपक जाती है (code बड़ा, speed ज़्यादा). Subroutine = एक copy, हर बार call/return (code छोटा, थोड़ा धीमा).

Q30. [Practice]

In C++, lines such as #include <iostream> and #define PI 3.14 are processed by the:

(A) Linker
(B) Preprocessor
(C) Loader
(D) Assembler

हर **option** समझो:

(A) Linker object files जोड़ता है, # lines नहीं देखता
(B) ✔ सही: # वाली lines (preprocessor directives) preprocessor process करता है
(C) Loader .exe को memory में लाता है
(D) Assembler assembly को machine code में बदलता है

Ans: (B) Preprocessor

Compiler से पहले preprocessor चलता है: #include header की text चिपकाता है और #define PI 3.14 में PI को 3.14 से बदल देता है (macro substitution).

Answer Key

Q1 - A	**Q2 - D**	**Q3 - C**	**Q4 - C**	**Q5 - B**
Q6 - B	**Q7 - C**	**Q8 - C**	**Q9 - A**	**Q10 - C**
Q11 - D	**Q12 - A**	**Q13 - A**	**Q14 - B**	**Q15 - D**
Q16 - A	**Q17 - B**	**Q18 - B**	**Q19 - B**	**Q20 - C**
Q21 - C	**Q22 - A**	**Q23 - C**	**Q24 - D**	**Q25 - B**
Q26 - A	**Q27 - C**	**Q28 - B**	**Q29 - A**	**Q30 - B**